WEEK 4 · LESSON 22 OF 24 · HANDOUT

Imbalanced Data and Fairness

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=6M2d2n-QXCc — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon Thailand: Addressing class imbalance in machine learning — https://www.youtube.com/watch?v=rDR0egRhJ1Y

🎓 **Go deeper:** PyCon 2019: Measuring Model Fairness — https://www.youtube.com/watch?v=pYXz507ths0

🎓 **Go deeper:** PyCon 2019: Measures and Mismeasures of Algorithmic Fairness — https://www.youtube.com/watch?v=olCKhigTVuY

Your goal in this lesson: Handle rare-event problems with the right metrics, class weights and thresholds, and check a model's results across groups of people for fairness.

Part 1 — When the positive class is rare

Fraud, disease outbreaks and machine failures are **rare**. Suppose only about **2%** of mobile-money transactions are fraud. A "model" that says "never fraud" is **98% accurate** — and worthless. This is the **accuracy paradox**.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X_f, y_f = make_classification(n_samples=20_000, n_features=8, n_informative=5,
                               weights=[0.98], class_sep=1.0, flip_y=0.005, random_state=42)
print("Fraud rate:", round(y_f.mean(), 4))
X_tr, X_te, y_tr, y_te = train_test_split(X_f, y_f, test_size=0.25, random_state=42,
stratify=y_f)
print("Accuracy of 'never fraud':", round(1 - y_te.mean(), 3))
```

Compare approaches with the right metrics

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_score, recall_score, average_precision_score

def report(name, model):
    model.fit(X_tr, y_tr)
    pred = model.predict(X_te)
    score = model.predict_proba(X_te)[:, 1]
    print(f"{name:28s} precision {precision_score(y_te, pred, zero_division=0):.2f}  "
          f"recall {recall_score(y_te, pred):.2f}  avg precision {average_precision_score(y_te,
score):.2f}")

report("Logistic regression", LogisticRegression())
report("Logistic, balanced weights", LogisticRegression(class_weight="balanced"))
report("Random forest", RandomForestClassifier(300, random_state=42, n_jobs=-1))
report("Random forest, balanced", RandomForestClassifier(300,
class_weight="balanced_subsample",
                                                         min_samples_leaf=5, random_state=42,
n_jobs=-1))
```

What you should see:

- **Plain logistic regression** flags **no** fraud at all (recall 0) — it learned that "no" is almost always right.
- **class_weight="balanced"** makes each fraud case count about 45 times more in training. Recall jumps to about **0.75** — but precision is only about **0.07**: 13 false alarms for every real fraud.
- **The random forest** finds the fraud pattern much better (average precision about **0.79**), catching about half the fraud with almost no false alarms; balanced weights raise its recall to about **0.58**.

Average precision (the area under the precision–recall curve) is a good single number for rare events: it ignores the huge number of easy negatives.

The tools for rare events

1. **Use the right metrics** — recall, precision, F1, average precision; never accuracy alone.
2. **Stratify** every split and cross-validation.
3. **Class weights** — `class_weight="balanced"` in most scikit-learn classifiers.
4. **Move the threshold** — choose it from costs (Lesson 15).
5. **Resampling** — undersample the majority class or oversample the minority (the `imbalanced-learn` library has SMOTE). Resample **only the training data**, never the test set.
6. **Get more positive examples** — often the best fix of all.

Part 2 — Is the model fair?

A model can be accurate overall and still treat groups of people differently. Let's add a **region** to the loans data. In our simulation, rural borrowers tend to earn less (as in many countries) — but the model is **not** given the region.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
rng = np.random.default_rng(21)
rural_chance = np.where(loans["income_rwf"] < 180_000, 0.6, 0.25)
loans["region"] = np.where(rng.random(len(loans)) < rural_chance, "rural", "urban")
print(loans.groupby("region")["defaulted"].agg(["count", "mean"]).round(3))
```

Train the usual model **without** the region, then audit it **by** region:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X = loans.drop(columns=["defaulted", "region"])
y, group = loans["defaulted"], loans["region"]
X_train, X_test, y_train, y_test, g_train, g_test = train_test_split(
    X, y, group, test_size=0.25, random_state=42, stratify=y)

model = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_train, y_train)
refuse = (model.predict_proba(X_test)[:, 1] >= 0.25).astype(int)

audit = pd.DataFrame({"actual": y_test.to_numpy(), "refused": refuse, "region":
g_test.to_numpy()})
def rates(df):
    good, bad = df[df["actual"] == 0], df[df["actual"] == 1]
    return pd.Series({"applicants": len(df),
                      "refusal rate": df["refused"].mean(),
                      "good borrowers refused": good["refused"].mean(),
                      "defaulters caught": bad["refused"].mean()})
print(audit.groupby("region")[["actual", "refused"]].apply(rates).round(2))
```

You should see that rural applicants are refused far more often (about **42%** vs **30%**), and that about **25%** of good rural borrowers are refused, compared with about **20%** of good urban borrowers.

Why — and is it unfair?

- The model never saw the region, but **income acts as a proxy**: removing a sensitive column does not remove its effect.
- Part of the gap reflects a real difference in default rates. Part is a heavier burden on good rural borrowers.

There is no single definition of fairness, and some cannot all be met at once:

Fairness idea	Asks	Here
Demographic parity	Are refusal rates similar across groups?	No: 42% vs 30%
Equal opportunity	Are good borrowers refused at similar rates?	Closer: 25% vs 20%
Calibration	Does a score of 0.3 mean the same risk in each group?	Check with real data

Deciding which matters is an **ethical and legal** question for the organisation, not only a technical one. Rwanda's data protection law (Law No. 058/2021) and the National AI Policy both expect AI systems to be responsible and accountable.

What you can do

- **Audit** every model by group before launch, and again regularly.
- Look for **proxies** and for missing or under-represented groups in the data.
- Adjust **thresholds** or retrain with better data if a group carries an unfair burden.
- Give people a **human appeal** and clear reasons for decisions.
- Document everything in a **model card** (next lesson).

Key terms

- **Imbalanced data** — one class is much rarer than the other.
- **Accuracy paradox** — high accuracy from always predicting the majority class.
- **Class weights** — making errors on the rare class count more.
- **Average precision** — area under the precision–recall curve.
- **Proxy** — a feature that indirectly reveals a sensitive attribute.
- **Demographic parity / equal opportunity** — two different fairness checks.

Remember this: For rare events, use recall, precision and average precision with class weights and cost-based thresholds; for people, audit results by group, because a model can be unfair even without sensitive columns.

Name: Date:

Activity

Task 1

For the plain fraud logistic regression, lower the threshold to 0.1. What happens to precision and recall?

Task 2

Undersample the training data so there are 5 normal transactions for every fraud, retrain logistic regression and compare.

Task 3

In the fairness audit, try thresholds 0.2 and 0.35. Does the gap between regions grow or shrink?

Task 4

Write a half-page memo to a SACCO board explaining the regional differences and what you recommend.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Only 2% of transactions are fraud. What accuracy does 'never fraud' get?
   A. 2%
   B. About 98%
   C. 50%
   D. 100%
2. What does class_weight='balanced' do?
   A. Deletes rare examples
   B. Makes errors on the rare class count more in training
   C. Balances the test set
   D. Sets the threshold to 0.5
3. Which metric is a good single number for rare-event problems?
   A. Number of rows
   B. Training error
   C. Accuracy
   D. Average precision (area under the precision–recall curve)
4. When resampling, which data should be resampled?
   A. Only the training data
   B. Both
   C. Neither
   D. Only the test data
5. The model never saw 'region' but treats regions differently. Why?
   A. Income acts as a proxy for region
   B. A bug in scikit-learn
   C. Random chance only
   D. The test set is too big
6. Equal opportunity asks…
   A. Are groups the same size?
   B. Are refusal rates the same in every group?
   C. Are good borrowers (true negatives) treated similarly across groups?
   D. Is accuracy above 90%?

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.